

TetMart Core

Website sản phẩm trang trí Tết

Python 3.x Django Backend MySQL Database Bootstrap Frontend

Mục lục

- 1. Tổng quan dự án
 - 2. Mục tiêu hệ thống
 - 3. Công nghệ sử dụng
 - 4. Kiến trúc tổng thể
 - 5. Cấu trúc thư mục
 - 6. Chức năng chính
 - 6.3. Chat Box hỗ trợ khách hàng
 - 7. Luồng xử lý nghiệp vụ
 - 8. Thiết kế dữ liệu
 - 9. Cài đặt và chạy dự án
 - 10. Cấu hình môi trường
 - 11. Quy ước phát triển
 - 12. Hướng phát triển tiếp theo
 - 13. Thành viên thực hiện
-

1. Tổng quan dự án

TetMart Core là hệ thống web thương mại điện tử mini chuyên cung cấp các sản phẩm trang trí Tết như đèn lồng, bao lì xì, vật phẩm trang trí, phụ kiện ngày Tết và các sản phẩm liên quan.

Dự án được xây dựng bằng **Django** theo hướng module hóa, dễ bảo trì và phù hợp với quy trình phát triển phần mềm thực tế. Hệ thống hỗ trợ hai nhóm người dùng chính:

- **Khách hàng:** xem sản phẩm, tìm kiếm, thêm giỏ hàng, đặt hàng, quản lý tài khoản và theo dõi đơn hàng.
 - **Quản trị viên / nhân sự nội bộ:** quản lý sản phẩm, đơn hàng, khách hàng, phân quyền và xem báo cáo thống kê.
-

2. Mục tiêu hệ thống

TetMart Core được phát triển với các mục tiêu chính:

- Xây dựng một website thương mại điện tử có đầy đủ luồng mua hàng cơ bản.
- Tách rõ logic theo từng module nghiệp vụ như sản phẩm, người dùng, giỏ hàng, đơn hàng và dashboard.
- Áp dụng mô hình **MVC/MVT của Django** để quản lý dữ liệu, xử lý nghiệp vụ và hiển thị giao diện.
- Hỗ trợ phân quyền quản trị để kiểm soát quyền truy cập từng khu vực trong dashboard.

- Thiết kế code dễ mở rộng cho các tính năng nâng cao như thanh toán online, quản lý tồn kho, API, thống kê nâng cao.
-

3. Công nghệ sử dụng

Backend

- **Python**: ngôn ngữ lập trình chính.
- **Django**: framework backend theo mô hình MVT.
- **Django Authentication**: xử lý đăng ký, đăng nhập, đăng xuất và phân quyền.
- **Django ORM**: thao tác với cơ sở dữ liệu thông qua model.

Frontend

- **HTML5**
- **CSS3**
- **Bootstrap**
- **JavaScript**
- **Django Template Engine**

Database

- **MySQL**: lưu trữ dữ liệu người dùng, sản phẩm, đơn hàng và chi tiết đơn hàng.

Thư viện / package chính

- Django
 - PyMySQL
 - Pillow
 - python-dotenv
-

4. Kiến trúc tổng thể

Dự án sử dụng kiến trúc **Django MVT**:

```
Client Browser
  |
  v
URL Dispatcher
  |
  v
View / Business Logic
  |
  v
Model / ORM
  |
  v
MySQL Database
```

|
v
Template Response

Trong đó:

- **URL Dispatcher** định tuyến request đến view tương ứng.
- **View** xử lý logic nghiệp vụ, kiểm tra quyền, truy vấn dữ liệu và trả về template.
- **Model** định nghĩa cấu trúc dữ liệu và quan hệ bảng.
- **Template** hiển thị dữ liệu ra giao diện người dùng.
- **Session** được dùng để lưu giỏ hàng tạm thời trước khi tạo đơn hàng.

5. Cấu trúc thư mục

```
TetMart-Core/
├── apps/
│   ├── cart/           # Xử lý giỏ hàng bằng session
│   └── dashboard/      # Một số chức năng dashboard mở rộng
├── orders/             # Quản lý đặt hàng, checkout, lịch sử đơn hàng
├── products/           # Quản lý danh mục, sản phẩm, trang sản phẩm
├── users/              # Người dùng, đăng ký, đăng nhập, tài khoản cá nhân
├── static/             # CSS, JavaScript, hình ảnh tĩnh
├── templates/          # Giao diện HTML dùng Django Template
├── tetmart/            # Cấu hình project Django
│   ├── settings.py
│   ├── urls.py
│   └── permission_utils.py # Helper kiểm tra phân quyền dashboard
├── database_backup.sql  # File backup database mẫu
├── manage.py           # Django management command
├── requirements.txt     # Danh sách dependencies
└── README.md
```

6. Chức năng chính

6.1. Chức năng dành cho khách hàng

Tài khoản người dùng

- Đăng ký tài khoản mới.
- Đăng nhập / đăng xuất.
- Tự động đăng nhập sau khi đăng ký thành công.
- Cập nhật thông tin cá nhân:
 - Họ tên
 - Email
 - Số điện thoại
 - Địa chỉ giao hàng

- Ảnh đại diện
- Mật khẩu

Sản phẩm

- Xem danh sách sản phẩm đang hoạt động.
- Xem chi tiết sản phẩm theo `slug`.
- Tìm kiếm sản phẩm theo tên.
- Lọc sản phẩm theo danh mục.
- Lọc theo khoảng giá.
- Sắp xếp theo sản phẩm mới nhất, giá tăng dần hoặc giá giảm dần.

Giỏ hàng

- Thêm sản phẩm vào giỏ hàng.
- Cập nhật số lượng sản phẩm.
- Xóa sản phẩm khỏi giỏ hàng.
- Xóa toàn bộ giỏ hàng.
- Tính tổng số lượng và tổng tiền.
- Lưu giỏ hàng bằng Django session.

Đặt hàng

- Mua ngay một sản phẩm.
- Checkout từ giỏ hàng.
- Tạo đơn hàng và chi tiết đơn hàng.
- Thanh toán khi nhận hàng (COD).
- Xem lịch sử đơn hàng.
- Hủy đơn hàng khi đơn còn ở trạng thái cho phép.
- Đặt lại đơn hàng đã hoàn thành hoặc đã hủy.

6.2. Chức năng dành cho quản trị viên

Dashboard tổng quan

- Tổng số đơn hàng.
- Tổng số sản phẩm.
- Tổng doanh thu từ các đơn hoàn thành.
- Thống kê đơn hàng theo trạng thái:
 - Đơn mới
 - Chờ xử lý
 - Đang giao
 - Hoàn thành
 - Đã hủy
- Hiển thị các đơn hàng gần nhất.

Quản lý sản phẩm

- Xem danh sách sản phẩm.
- Tìm kiếm sản phẩm theo tên, slug hoặc mã sản phẩm.
- Lọc theo danh mục.
- Lọc theo trạng thái còn hàng / hết hàng.
- Sắp xếp theo thời gian hoặc giá.
- Thêm sản phẩm mới.
- Cập nhật thông tin sản phẩm.
- Tự động tạo **slug** từ tên sản phẩm.
- Xóa sản phẩm.

Quản lý đơn hàng

- Xem danh sách đơn hàng.
- Tìm kiếm theo mã đơn hàng, tên khách hàng hoặc số điện thoại.
- Lọc theo trạng thái đơn hàng.
- Xem chi tiết đơn hàng.
- Cập nhật trạng thái đơn hàng.

Quản lý khách hàng

- Xem danh sách khách hàng.
- Tìm kiếm theo tên đăng nhập, email hoặc số điện thoại.
- Thống kê số đơn và tổng chi tiêu của từng khách hàng.
- Phân loại khách hàng theo trạng thái:
 - Mới
 - Đang hoạt động
 - VIP
 - Không hoạt động

Báo cáo doanh thu

- Xem doanh thu theo khoảng ngày.
- Thống kê số đơn hoàn thành.
- Tính tỷ lệ hủy đơn.
- Thống kê doanh thu theo ngày.
- Thống kê số đơn theo ngày.
- Thống kê doanh thu theo danh mục.
- Hiển thị top sản phẩm bán chạy.

Phân quyền

Hệ thống có lớp kiểm tra quyền riêng trong **permission_utils.py**, hỗ trợ điều hướng người dùng đến dashboard phù hợp với quyền hiện có.

Các nhóm quyền chính:

- Quyền xem dashboard.
- Quyền quản lý đơn hàng.

- Quyền quản lý khách hàng.
- Quyền quản lý sản phẩm.
- Quyền xem báo cáo.
- Quyền quản lý phân quyền.

6.3. Chat Box hỗ trợ khách hàng

Hệ thống tích hợp **Tawk.to** để cung cấp chat box trực tuyến giữa khách hàng và quản trị viên.

- Chat box được nhúng trực tiếp vào giao diện website thông qua script của Tawk.to.
- Khách hàng có thể gửi tin nhắn hỗ trợ ngay trên website.
- Quản trị viên tiếp nhận và phản hồi tin nhắn thông qua dashboard của Tawk.to.
- Không cần xây dựng backend chat riêng, giúp giảm độ phức tạp hệ thống và triển khai nhanh hơn.

7. Luồng xử lý nghiệp vụ

7.1. Luồng đăng ký / đăng nhập

```
graph TD
    A[Người dùng nhập form] --> B[Django Form validate dữ liệu]
    B --> C[Tạo User / xác thực User]
    C --> D[Đăng nhập vào hệ thống]
    D --> E[Điều hướng về trang chủ hoặc dashboard phù hợp]
```

7.2. Luồng thêm sản phẩm vào giỏ hàng

```
graph TD
    A[Khách hàng chọn sản phẩm] --> B[Gửi request POST đến cart/add]
    B --> C[Lấy sản phẩm từ database]
    C --> D[Lưu hoặc cập nhật sản phẩm trong session cart]
    D --> E[Trả về số lượng sản phẩm hiện có trong giỏ]
```

7.3. Luồng checkout từ giỏ hàng

```
graph TD; A[Khách hàng mở trang checkout] --> B[ ]; B -- v --> C[Hệ thống đọc cart từ session]; C --> D[ ]; D -- v --> E[Truy vấn sản phẩm tương ứng trong database]; E --> F[ ]; F -- v --> G[Tính tổng số lượng và tổng tiền]; G --> H[ ]; H -- v --> I[Hiển thị form xác nhận thông tin giao hàng];
```

7.4. Luồng tạo đơn hàng

```
graph TD; A[Khách hàng xác nhận đặt hàng] --> B[ ]; B -- v --> C[Tạo Order với trạng thái mặc định: new]; C --> D[ ]; D -- v --> E[Tạo các OrderItem tương ứng]; E --> F[ ]; F -- v --> G[Tính tổng giá trị đơn hàng]; G --> H[ ]; H -- v --> I[Lưu đơn hàng]; I --> J[ ]; J -- v --> K[Xóa dữ liệu giỏ hàng / checkout session]; K --> L[ ]; L -- v --> M[Chuyển đến trang đơn hàng của tôi];
```

7.5. Luồng quản trị đơn hàng

```
graph TD; A[Admin truy cập danh sách đơn hàng] --> B[ ]; B -- v --> C[Hệ thống kiểm tra quyền orders.view_order]; C --> D[ ]; D -- v --> E[Admin xem chi tiết đơn hàng]; E --> F[ ];
```

```
      v
Nếu có quyền orders.update_order_status
      |
      v
Admin cập nhật trạng thái đơn
      |
      v
Lưu trạng thái mới vào database
```

8. Thiết kế dữ liệu

8.1. User

Model **User** kế thừa từ **AbstractUser** và mở rộng thêm các trường:

Trường	Ý nghĩa
phone	Số điện thoại
address	Địa chỉ giao hàng
avatar	Ảnh đại diện
is_customer	Đánh dấu tài khoản là khách hàng

8.2. Category

Trường	Ý nghĩa
name	Tên danh mục
slug	Đường dẫn SEO duy nhất

8.3. Product

Trường	Ý nghĩa
category	Danh mục sản phẩm
name	Tên sản phẩm
slug	Đường dẫn SEO duy nhất
image	Ảnh sản phẩm
description	Mô tả sản phẩm
price	Giá bán
stock	Số lượng tồn kho
is_active	Trạng thái đang bán
created_at	Ngày tạo

8.4. Order

Trường	Ý nghĩa
<code>user</code>	Người đặt hàng
<code>full_name</code>	Họ tên người nhận
<code>phone</code>	Số điện thoại nhận hàng
<code>address</code>	Địa chỉ giao hàng
<code>note</code>	Ghi chú
<code>total_price</code>	Tổng giá trị đơn hàng
<code>status</code>	Trạng thái đơn hàng
<code>created_at</code>	Ngày đặt hàng

Trạng thái đơn hàng:

Mã trạng thái	Ý nghĩa
<code>new</code>	Đơn mới
<code>pending</code>	Chờ xử lý
<code>shipping</code>	Đang giao
<code>completed</code>	Hoàn thành
<code>cancelled</code>	Đã hủy

8.5. OrderItem

Trường	Ý nghĩa
<code>order</code>	Đơn hàng cha
<code>product</code>	Sản phẩm được mua
<code>price</code>	Giá sản phẩm tại thời điểm mua
<code>quantity</code>	Số lượng mua
<code>total</code>	Thành tiền của dòng sản phẩm

9. Cài đặt và chạy dự án

9.1. Clone repository

```
git clone https://github.com/sonle-dev/TetMart-Core.git
cd TetMart-Core
```

```
git checkout feature/backend
```

9.2. Tạo môi trường ảo

Windows

```
python -m venv .venv  
.venv\Scripts\activate
```

Linux / macOS

```
python3 -m venv .venv  
source .venv/bin/activate
```

9.3. Cài đặt dependencies

```
pip install -r requirements.txt
```

9.4. Tạo database MySQL

```
CREATE DATABASE tet_mart_db  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;
```

9.5. Cấu hình database

Cập nhật cấu hình trong `tetmart/settings.py` hoặc chuyển sang dùng biến môi trường trong file `.env`:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'tet_mart_db',  
        'USER': 'your_mysql_user',  
        'PASSWORD': 'your_mysql_password',  
        'HOST': '127.0.0.1',  
        'PORT': '3306',  
    }  
}
```

9.6. Chạy migration

```
python manage.py makemigrations
python manage.py migrate
```

9.7. Import dữ liệu mẫu nếu cần

```
mysql -u your_mysql_user -p tet_mart_db < database_backup.sql
```

9.8. Tạo tài khoản quản trị

```
python manage.py createsuperuser
```

9.9. Chạy server

```
python manage.py runserver
```

Truy cập ứng dụng tại:

```
http://127.0.0.1:8000/
```

Trang quản trị Django:

```
http://127.0.0.1:8000/admin/
```

10. Cấu hình môi trường

Dự án đã import `python-dotenv`, vì vậy nên dùng file `.env` để tách cấu hình nhạy cảm khỏi source code.

Ví dụ file `.env`:

```
SECRET_KEY=your-secret-key
DEBUG=True
DB_NAME=tet_mart_db
DB_USER=your_mysql_user
DB_PASSWORD=your_mysql_password
DB_HOST=127.0.0.1
DB_PORT=3306
```

Khuyến nghị khi triển khai:

- Không commit file `.env`.
- Không hard-code `SECRET_KEY`, tài khoản database hoặc mật khẩu trong source code.
- Tắt `DEBUG` khi deploy production.
- Cấu hình `ALLOWED_HOSTS` theo domain thực tế.
- Backup database định kỳ.

11. Quy ước phát triển

11.1. Git branch workflow

```
feature/* -> develop -> main
```

Các nhánh thường dùng:

- `main`: phiên bản ổn định.
- `develop`: tích hợp các tính năng đã hoàn thiện.
- `feature/backend`: phát triển backend.
- `feature/frontend`: phát triển giao diện.
- `feature/tester`: kiểm thử.

11.2. Quy ước commit

Sử dụng format commit rõ ràng:

```
git commit -m "feat: thêm chức năng lọc sản phẩm theo giá"
git commit -m "fix: sửa lỗi đăng nhập không chuyển trang"
git commit -m "refactor: tối ưu logic tạo đơn hàng"
git commit -m "docs: cập nhật hướng dẫn cài đặt"
```

Một số prefix nên dùng:

Prefix	Ý nghĩa
<code>feat</code>	Thêm tính năng mới
<code>fix</code>	Sửa lỗi
<code>refactor</code>	Tái cấu trúc code
<code>docs</code>	Cập nhật tài liệu
<code>style</code>	Chỉnh format, UI, convention
<code>test</code>	Thêm hoặc sửa test
<code>chore</code>	Công việc phụ trợ

11.3. Quy tắc code

- Tách logic theo app đúng trách nhiệm.
 - Đặt tên biến, hàm, class rõ nghĩa.
 - Không viết logic xử lý quá dài trong template.
 - Kiểm tra quyền trước khi truy cập dashboard.
 - Validate dữ liệu đầu vào trước khi lưu database.
 - Không commit file chứa thông tin nhạy cảm.
 - Kiểm tra migration trước khi push code.
-

12. Hướng phát triển tiếp theo

Một số tính năng có thể mở rộng:

- Xây dựng REST API bằng Django REST Framework.
 - Tích hợp thanh toán online như VNPay, MoMo hoặc ZaloPay.
 - Tự động trừ tiền kho khi đơn hàng được xác nhận.
 - Gửi email xác nhận đơn hàng.
 - Thêm chức năng đánh giá sản phẩm.
 - Thêm wishlist / sản phẩm yêu thích.
 - Thêm dashboard biểu đồ trực quan bằng Chart.js.
 - Thêm unit test và integration test.
 - Docker hóa môi trường chạy dự án.
 - Triển khai lên VPS, Render, Railway hoặc cloud platform.
-

13. Thành viên thực hiện

- **Lê Hồng Sơn**
 - **Nguyễn Danh Thế**
 - **Ngô Thị Sinh**
-

Ghi chú

Dự án được xây dựng phục vụ mục đích học tập, thực hành quy trình phát triển phần mềm và mô phỏng một hệ thống thương mại điện tử thực tế quy mô nhỏ.